UNIVERSIDADE TUIUTI DO PARANÁ

FACULDADE DE CIÊNCIAS EXATAS E DE TECNOLOGIA

BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

LUCAS MÜLLER SCUZZIATO

EXPRESSÕES REGULARES (REGEX): CONCEITOS, PROGRAMAÇÃO EM PYTHON E APLICAÇÕES

ESTUDO DIRIGIDO — 1º BIMESTRE

TEORIA DA COMPUTAÇÃO E LINGUAGENS FORMAIS

PROF. DIÓGENES COGO FURLAN

CURITIBA

2026

SUMÁRIO

1 O QUE É REGEX 2
1.1 CARACTERES ESPECIAIS 2
1.2 CARACTERES DE AGRUPAMENTO 2
1.3 CARACTERES ÂNCORA 2
1.4 QUANTIFICADORES 3
1.5 CLASSES DE CARACTERES 4
2 PROGRAMAÇÃO COM A BIBLIOTECA RE 6
2.1 MÉTODOS PRINCIPAIS 6
2.2 GRUPOS, SUBSTITUIÇÃO E FLAGS 7
3 DEZ PROBLEMAS RESOLVIDOS 8
3.1 VALIDAR CPF PELO FORMATO 8
3.2 VALIDAR E-MAIL 8
3.3 EXTRAIR TELEFONES DE UM TEXTO 9
3.4 VALIDAR PLACA NO PADRÃO MERCOSUL 10
3.5 EXTRAIR DATAS NO FORMATO DIA/MÊS/ANO 10
3.6 VALIDAR CEP 11
3.7 SENHA FORTE 11
3.8 REMOVER ESPAÇOS EXTRAS 12
3.9 EXTRAIR URLS DE UM TEXTO 12
3.10 MASCARAR O NÚMERO DO CARTÃO 13
APÊNDICE A CÓDIGO-FONTE COMPLETO 14
REFERÊNCIAS 19

1 O QUE É REGEX

Uma expressão regular (REGEX) é uma sequência de caracteres que descreve um padrão de busca em um texto. Em vez de procurar uma cadeia exata, como `123.456.789-00`, escreve-se um formato capaz de reconhecer toda cadeia daquele tipo. O padrão é a soma de literais, que são os caracteres fixos, com metacaracteres, que indicam repetição, alternativa, posição e classe (DevMedia, s.d.).

Na disciplina, esse recurso é a versão prática das linguagens regulares. A barra vertical corresponde à união, a justaposição corresponde à concatenação, o asterisco corresponde ao fecho de Kleene e o ponto de interrogação corresponde à escolha entre o símbolo e a cadeia vazia. Os exemplos deste trabalho usam a notação da biblioteca `re` do Python (Python Software Foundation, s.d.), que é a forma dessas expressões nas ferramentas de programação.

1.1 CARACTERES ESPECIAIS

Metacaractere é o símbolo que deixa de valer como letra e passa a valer como operador do padrão. Quando o símbolo precisa aparecer como texto, usa-se a barra invertida. Assim, `\.` casa só o ponto, e não um caractere qualquer.

A expressão `(ab)+` descreve a mesma linguagem que `(ab)(ab)*`. A alternância `a|b` é a união, e `a?` equivale a escolher `a` ou a cadeia vazia.

1.2 CARACTERES DE AGRUPAMENTO

Os parênteses juntam vários símbolos num único elemento, para que um quantificador se aplique ao bloco inteiro. Na biblioteca `re`, o parêntese comum também captura o trecho, e esse trecho pode ser reutilizado na substituição ou por uma referência numérica.

1.3 CARACTERES ÂNCORA

Âncora não consome caractere. Ela exige que o padrão ocorra numa posição: início, fim, fronteira de palavra ou, no caso dos *lookaheads*, um contexto à frente ou atrás.

O problema da senha forte, no Capítulo 3, usa *lookahead* para exigir maiúscula,

QUADRO 1 – CARACTERES ESPECIAIS (METACARACTERES)

Símbolo	**Significado**	**Exemplo**	**Casa com**
`.`	Qualquer caractere, exceto quebra de linha	`a.c`	`abc`, `a1c`, `a c`
`*`	Zero ou mais repetições do elemento anterior	`ab*c`	`ac`, `abc`, `abbc`
`+`	Uma ou mais repetições	`ab+c`	`abc`, `abbc`
`?`	Zero ou uma vez	`ab?c`	`ac`, `abc`
`{n}`	Exatamente *n* repetições	`\d{3}`	`123`
`\`	Escape: o caractere seguinte é literal	`\.`	só o ponto
`|`	Alternância (união)	`a|b`	`a` ou `b`
`()`	Agrupamento e captura	`(ab)+`	`ab`, `abab`
`[]`	Classe de caracteres	`[abc]`	um entre `a`, `b` e `c`
`^`	Início da string	`^abc`	só se começa com `abc`
`$`	Fim da string	`abc$`	só se termina com `abc`

FONTE: Elaborado pelo autor, com base em DevMedia (s.d.).

QUADRO 2 – AGRUPAMENTO E CAPTURA

Símbolo	**Papel**	**Exemplo**
`(...)`	Agrupa e captura o trecho	`(\d{2})/(\d{2})`
`(?:...)`	Agrupa sem capturar	`(?:ab)+`
`(?P<nome>...)`	Grupo nomeado, específico do Python	`(?P<ano>\d{4})`
`\1`, `\2`	Referência ao grupo já capturado	`(\w)\1` casa `aa`, `bb`

FONTE: Autoria própria

minúscula e dígito sem fixar a ordem desses caracteres. O mascaramento de cartão usa *lookbehind* e *lookahead* negativos para não cortar um número maior no meio.

1.4 QUANTIFICADORES

O quantificador diz quantas vezes o elemento anterior pode se repetir. Por padrão ele é guloso: consome o máximo que ainda permite o restante do padrão casar. A forma preguiçosa, com um ponto de interrogação depois do quantificador,

QUADRO 3 – ÂNCORAS

Âncora	**Significado**
`^`	Início da string. Com a flag `re.MULTILINE`, início de cada linha.
`$`	Fim da string. Com `re.MULTILINE`, fim de cada linha.
`\b`	Fronteira de palavra, entre um caractere de palavra e um que não é.
`\B`	Posição que não é fronteira de palavra.
`(?=...)`	*Lookahead*: exige o padrão adiante, sem consumi-lo.
`(?!...)`	*Lookahead* negativo.
`(?<=...)`	*Lookbehind*: exige algo imediatamente antes, sem consumi-lo.
`(?<!...)`	*Lookbehind* negativo.

FONTE: Autoria própria

consome o mínimo.

QUADRO 4 – QUANTIFICADORES E A NOTAÇÃO DA DISCIPLINA

Símbolo	**Repetições**	**Leitura na disciplina**
`*`	Zero ou mais	Fecho de Kleene
`+`	Uma ou mais	O símbolo seguido do fecho
`?`	Zero ou uma	O símbolo ou a cadeia vazia
`{n}`	Exatamente *n*	Concatenação repetida *n* vezes
`{n,}`	*n* ou mais	*n* símbolos e depois o fecho
`{n,m}`	Entre *n* e *m*	União das concatenações desse intervalo
`*?`	Mínimo, não o máximo	Forma preguiçosa do fecho

FONTE: Autoria própria

Um caso clássico é a marcação `<.* >` escrita sem espaço, isto é, `<.*>`: o ponto-asterisco guloso avança até o último `>` da linha. A forma `<.*?>` para no primeiro.

1.5 CLASSES DE CARACTERES

A classe escolhe um caractere dentre um conjunto. O circunflexo logo após o colchete nega a classe. Fora dos colchetes, o mesmo circunflexo é a âncora de início, vista na Seção 1.3.

No Python 3, sem a flag `re.ASCII`, `\w` também casa letras Unicode, inclusive acentuadas. A equivalência com `[A-Za-z0-9_]` vale no modo ASCII. Os exemplos

QUADRO 5 – CLASSES DE CARACTERES

Classe	**Significado**	**Equivalência usual**
`[abc]`	Um dos caracteres listados	união de `a`, `b` e `c`
`[a-z], [0-9]`	Uma faixa	—
`[^abc]`	Qualquer caractere exceto `a`, `b` e `c`	—
`\d`	Dígito	`[0-9]`
`\D`	O que não é dígito	`[^0-9]`
`\w`	Caractere de palavra	`[A-Za-z0-9_]` no modo ASCII
`\W`	O que não é caractere de palavra	complemento de `\w`
`\s`	Espaço em branco	espaço, tabulação e quebra de linha
`\S`	O que não é espaço em branco	complemento de `\s`

FONTE: Elaborado pelo autor, com base em DevMedia (s.d.), Python Software Foundation (s.d.).

deste trabalho usam textos sem acento nos trechos casados, então as duas leituras coincidem.

2 PROGRAMAÇÃO COM A BIBLIOTECA RE

O Python traz as expressões regulares na biblioteca padrão `re`, carregada com `import re` (Python Software Foundation, s.d.). Não há pacote externo. O texto usado nos métodos abaixo é `O curso custa R$ 1999.90 e o outro R$ 350.00.` e o padrão dos valores é `\d+\.\d{2}`.

2.1 MÉTODOS PRINCIPAIS

Cada método devolve um objeto de casamento, uma lista ou uma string nova. Quando não há casamento, `search`, `match` e `fullmatch` devolvem `None`.

QUADRO 6 – MÉTODOS DO MÓDULO RE USADOS NESTE TRABALHO

Método	**O que faz**
`re.search`	Procura o padrão em qualquer posição e devolve o primeiro casamento.
`re.match`	Igual ao `search`, mas só a partir do início da string.
`re.fullmatch`	Exige que a string inteira case com o padrão.
`re.findall`	Devolve a lista de todos os casamentos.
`re.finditer`	Igual ao `findall`, como iterador, com acesso ao intervalo (*span*).
`re.sub`	Substitui cada casamento por outro texto.
`re.split`	Divide a string nos pontos em que o padrão casa.
`re.compile`	Compila o padrão uma vez para reutilizá-lo, com ou sem flags.

FONTE: Elaborado pelo autor, com base em Python Software Foundation (s.d.).

O trecho a seguir foi executado com Python 3. A diferença entre `match` e `search` aparece na string `custa 1999.90`: o valor não está no início, então `match` devolve `None` e `search` devolve `1999.90`.

```
1  import re
2
3  texto = "O curso custa R$ 1999.90 e o outro R$ 350.00."
4  m = re.search(r"\d+\.\d{2}", texto)
5  print(m.group(), m.span())
6  print(re.match(r"\d+\.\d{2}", "1999.90 reais").group())
7  print(re.match(r"\d+\.\d{2}", "custa 1999.90"))
8  print(re.search(r"\d+\.\d{2}", "custa 1999.90").group())
```

```
print(re.findall(r"\d+\.\d{2}", texto))
print(re.sub(r"\s+", " ", "um   texto   com   espacos   extras."))
print(re.split(r"[;,]", "a;b,c;d"))
```

```
1999.90 (17, 24)
1999.90
None
1999.90
['1999.90', '350.00']
um texto com espacos extras.
['a', 'b', 'c', 'd']
```

`finditer` devolve os mesmos dois valores com as posições `(17, 24)` e `(38, 44)`. `fullmatch` no CPF `123.456.789-00`, com o padrão do Capítulo 3, devolve a própria string. `compile` seguido de `findall` repete a lista `['1999.90', '350.00']`.

2.2 GRUPOS, SUBSTITUIÇÃO E FLAGS

O grupo nomeado entra na string de substituição com `\g<nome>`. As flags mudam a leitura do padrão sem alterar os metacaracteres: `re.IGNORECASE` ignora maiúsculas e minúsculas, e `re.MULTILINE` faz `^` e `$` valerem em cada linha.

```
import re

print(re.sub(
    r"(?P<ano>\d{4})-(?P<mes>\d{2})",
    r"\g<mes>/\g<ano>",
    "2026-08",
))
print(re.findall(r"ab[cs]", "Abc abC aBc", re.IGNORECASE))
print(re.findall(r"^\w+", "linha1\nlinha2\nlinha3", re.MULTILINE))
```

```
08/2026
['Abc', 'abC', 'aBc']
['linha1', 'linha2', 'linha3']
```

3 DEZ PROBLEMAS RESOLVIDOS

Cada problema traz a entrada, o padrão, o trecho em Python e a saída obtida na mesma execução do script `ed1-regex-demo.py`, reproduzido no Apêndice A. Todos usam `re.fullmatch` quando a string inteira precisa ser válida, e `re.findall` ou `re.sub` quando a tarefa é extrair ou reescrever um trecho dentro de um texto maior.

3.1 VALIDAR CPF PELO FORMATO

O padrão aceita só duas formas completas: a pontuação oficial, com pontos e hífen, ou os onze dígitos seguidos. Separadores opcionais e independentes aceitariam misturas como `123.456789-00`, que não é nenhum dos dois formatos. Este trabalho não verifica os dígitos verificadores, apenas a forma.

```
^(\d{3}\.\d{3}\.\d{3}-\d{2}|\d{11})$
```

```
import re
pat = r"^(\d{3}\.\d{3}\.\d{3}-\d{2}|\d{11})$"
for item in ["123.456.789-00", "12345678900", "12.3.456.789-00",
             "123.456789-00", "ABC.DEF.GHI-JK"]:
    print(item, "VALIDO" if re.fullmatch(pat, item) else "INVALIDO")
```

```
123.456.789-00 VALIDO
12345678900 VALIDO
12.3.456.789-00 INVALIDO
123.456789-00 INVALIDO
ABC.DEF.GHI-JK INVALIDO
```

3.2 VALIDAR E-MAIL

O padrão exige um nome, o arroba, um domínio com pelo menos um ponto e um sufixo de duas ou mais letras. Não é a gramática completa de e-mail da internet. Cobre os casos úteis deste exercício.

```
^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$
```

```
import re
pat = r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"
for item in ["joao@abc.com", "vendas@loja.com.br", "invalido@",
             "sem-arroba.com", "ana@site.org.br"]:
    print(item, "VALIDO" if re.fullmatch(pat, item) else "INVALIDO")
```

```
joao@abc.com VALIDO
vendas@loja.com.br VALIDO
invalido@ INVALIDO
sem-arroba.com INVALIDO
ana@site.org.br VALIDO
```

3.3 EXTRAIR TELEFONES DE UM TEXTO

A entrada mistura celular com DDD entre parênteses, fixo com DDD solto e um terceiro número em que o espaço separa os blocos finais.

```
Fale com Joao no (41) 99876-5432 ou com Maria no 41 3322-1000. Fixo: (45)
    9812 2233.
```

A primeira alternativa lê o DDD entre parênteses, um bloco de quatro ou cinco dígitos e mais quatro. A segunda lê o DDD sem parênteses, separado por espaço.

```
\(\d{2}\)\s?\d{4,5}[ -]?\d{4}|\d{2}\s\d{4}[ -]?\d{4}
```

```
import re
texto = ("Fale com Joao no (41) 99876-5432 ou com Maria no 41 3322-1000. "
         "Fixo: (45)9812 2233.")
pat = r"\(\d{2}\)\s?\d{4,5}[ -]?\d{4}|\d{2}\s\d{4}[ -]?\d{4}"
print(re.findall(pat, texto))
```

```
['(41) 99876-5432', '41 3322-1000', '(45)9812 2233']
```

3.4 VALIDAR PLACA NO PADRÃO MERCOSUL

A placa Mercosul brasileira tem sete posições: três letras, um dígito, uma letra e dois dígitos, sem hífen. A quinta posição é letra. Um padrão que aceitasse dígito nessa posição marcaria `ABC1234` como válida, e essa cadeia é o modelo antigo sem hífen, não o Mercosul.

```
^[A-Z]{3}\d[A-Z]\d{2}$
```

```
import re
pat = r"^[A-Z]{3}\d[A-Z]\d{2}$"
for item in ["ABC1D23", "ABC-1234", "ABC1234", "ABC1D2X", "AB1C2D3", "
    XYZ9A99"]:
     print(item, "VALIDA" if re.fullmatch(pat, item) else "INVALIDA")
```

```
ABC1D23 VALIDA
ABC-1234 INVALIDA
ABC1234 INVALIDA
ABC1D2X INVALIDA
AB1C2D3 INVALIDA
XYZ9A99 VALIDA
```

3.5 EXTRAIR DATAS NO FORMATO DIA/MÊS/ANO

O texto de entrada é: `Prova dia 04/09/2026, entrega em 11/9/2026 e feriado 7/09. Sem data 99-99-9999.`

```
\b\d{1,2}/\d{1,2}/\d{4}\b
```

```
import re
texto = ("Prova dia 04/09/2026, entrega em 11/9/2026 e feriado 7/09. "
         "Sem data 99-99-9999.")
print(re.findall(r"\b\d{1,2}/\d{1,2}/\d{4}\b", texto))
```

```
['04/09/2026', '11/9/2026']
```

`99-99-9999` fica de fora porque o padrão exige a barra, não o hífen. `7/09` fica de fora porque não há ano com quatro dígitos. A âncora `\b` tem outro papel: impedir que a data seja lida no meio de uma palavra. Ela não é o que rejeita a cadeia com hífen.

3.6 VALIDAR CEP

O CEP tem oito dígitos. O hífen depois do quinto é opcional, mas, se aparecer, precisa estar nessa posição. `8120-0100` tem o hífen no lugar errado e é recusado.

```
^\d{5}-?\d{3}$
```

```
import re
pat = r"^\d{5}-?\d{3}$"
for item in ["81200-100", "81200100", "8120-0100", "8120010"]:
    print(item, "VALIDO" if re.fullmatch(pat, item) else "INVALIDO")
```

```
81200-100 VALIDO
81200100 VALIDO
8120-0100 INVALIDO
8120010 INVALIDO
```

3.7 SENHA FORTE

A senha precisa ter pelo menos oito caracteres, uma maiúscula, uma minúscula e um dígito. Três *lookaheads* testam essas exigências sem consumir a string. O `.{8,}` só então lê o tamanho. A ordem das letras dentro da senha deixa de importar.

```
^(?=.*[A-Z])(?=.*[a-z])(?=.*\d).{8,}$
```

```
import re
pat = r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d).{8,}$"
for item in ["abc123", "SenhaForte1", "SENHA123", "senha123", "S3nha.Forte
    !"]:
    print(item, "OK" if re.fullmatch(pat, item) else "FALHOU")
```

```
abc123 FALHOU
SenhaForte1 OK
SENHA123 FALHOU
senha123 FALHOU
S3nha.Forte! OK
```

`abc123` falha pelo tamanho e pela falta de maiúscula. `SENHA123` não tem minúscula. `senha123` não tem maiúscula.

3.8 REMOVER ESPAÇOS EXTRAS

A entrada repete espaços no meio da frase e ainda deixa espaços no final, como mostra o código abaixo. `\s+` casa cada sequência de branco. `re.sub` troca essa sequência por um espaço só. O `strip` tira o espaço que sobra nas pontas. Sem o `strip`, a frase limpa ainda terminaria em espaço.

```
import re
entrada = "A  aula    de   Regex   sera  as   10h,    na sala   B-2.   "
print(re.sub(r"\s+", " ", entrada).strip())
```

```
A aula de Regex sera as 10h, na sala B-2.
```

3.9 EXTRAIR URLS DE UM TEXTO

O padrão exige o esquema `http` ou `https`. Por isso `www.uol.com.br`, presente na entrada, não entra no resultado. A classe final evita puxar o ponto que apenas encerra a frase.

```
https?://[\w.-]+\.\w+(?:[/?#][^\s.,;]*)?
```

```
import re
entrada = ("Veja https://www.devmedia.com.br/iniciando-expressoes-
    regulares/6557 "
           "e http://example.com.br/aula?x=1. Site: www.uol.com.br")
pat = r"https?://[\w.-]+\.\w+(?:[/?#][^\s.,;]*)?"
print(re.findall(pat, entrada))
```

```
['https://www.devmedia.com.br/iniciando-expressoes-regulares/6557', 'http
    ://example.com.br/aula?x=1']
```

3.10 MASCARAR O NÚMERO DO CARTÃO

A entrada traz dois cartões, um separado por espaços e outro por hífens. Quatro grupos capturam os blocos. A substituição conserva só o quarto grupo, que são os quatro últimos dígitos. O *lookbehind* e o *lookahead* negativos exigem que o bloco não esteja colado em outro dígito.

```
(?<!\d)(\d{4})[ -]?(\d{4})[ -]?(\d{4})[ -]?(\d{4})(?!\d)
```

```
import re
entrada = "Cartao 5123 4678 9012 3456 aprovado; outro 9876-5432-1098-7654
    negado."
pat = r"(?<!\d)(\d{4})[ -]?(\d{4})[ -]?(\d{4})[ -]?(\d{4})(?!\d)"
print(re.sub(pat, r"**** **** **** \4", entrada))
```

```
Cartao **** **** **** 3456 aprovado; outro **** **** **** 7654 negado.
```

Os dez problemas usam só os operadores do Capítulo 1: classe, quantificador, agrupamento, alternância e âncora. A programação se resume aos métodos do Capítulo 2. O que muda de um problema para o outro é o padrão, não a biblioteca.

APÊNDICE A – CÓDIGO-FONTE COMPLETO

O arquivo `material/exercicios/ed1-regex-demo.py` gera, numa única execução, as saídas citadas nos Capítulos 2 e 3.

```python
"""ED1 - expressoes regulares. Gera as saidas usadas no relatorio."""

import re

texto = "O curso custa R$ 1999.90 e o outro R$ 350.00."


def titulo(nome):
    print()
    print("=" * 64)
    print(nome)
    print("=" * 64)


titulo("METODOS DA BIBLIOTECA re")
print("texto:", texto)

m = re.search(r"\d+\.\d{2}", texto)
print("search:", None if m is None else f"{m.group()} | span {m.span()}")
print("search sem match:", re.search(r"\d+\.\d{2}", "sem numero aqui"))

m = re.match(r"\d+\.\d{2}", "1999.90 reais")
print("match no inicio:", None if m is None else m.group())
m = re.match(r"\d+\.\d{2}", "custa 1999.90")
print("match no meio:", None if m is None else m.group())
m = re.search(r"\d+\.\d{2}", "custa 1999.90")
print("search no meio:", None if m is None else m.group())

m = re.fullmatch(r"^(\d{3}\.\d{3}\.\d{3}-\d{2}|\d{11})$", "123.456.789-00"
    )
print("fullmatch:", None if m is None else m.group())
```

```
31
32  print("findall:", re.findall(r"\d+\.\d{2}", texto))
33  print("finditer:", [(x.group(), x.span()) for x in re.finditer(r"\d+\.\d
        {2}", texto)])
34  print("sub:", re.sub(r"\s+", " ", "um   texto   com   espacos  extras."))
35  print("split:", re.split(r"[;,]", "a;b,c;d"))
36
37  padrao = re.compile(r"\d+\.\d{2}")
38  print("compile:", padrao.findall(texto))
39  print(
40      "grupos nomeados:",
41      re.sub(r"(?P<ano>\d{4})-(?P<mes>\d{2})", r"\g<mes>/\g<ano>", "2026-08"
        ),
42  )
43  print("IGNORECASE:", re.findall(r"ab[cs]", "Abc abC aBc", re.IGNORECASE))
44  print("MULTILINE:", re.findall(r"^\w+", "linha1\nlinha2\nlinha3", re.
        MULTILINE))
45
46
47  titulo("1) VALIDAR CPF (formato)")
48  entrada = [
49      "123.456.789-00",
50      "12345678900",
51      "12.3.456.789-00",
52      "123.456789-00",
53      "ABC.DEF.GHI-JK",
54  ]
55  pat = r"^(\d{3}\.\d{3}\.\d{3}-\d{2}|\d{11})$"
56  print("regex:", pat)
57  for item in entrada:
58      print(f"  {item!r:22} -> {'VALIDO' if re.fullmatch(pat, item) else '
        INVALIDO'}")
59
60
61  titulo("2) VALIDAR E-MAIL")
```

```
entrada = [
    "joao@abc.com",
    "vendas@loja.com.br",
    "invalido@",
    "sem-arroba.com",
    "ana@site.org.br",
]
pat = r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"
print("regex:", pat)
for item in entrada:
    print(f"  {item!r:22} -> {'VALIDO' if re.fullmatch(pat, item) else '
    INVALIDO'}")


titulo("3) EXTRAIR TELEFONES")
entrada = "Fale com Joao no (41) 99876-5432 ou com Maria no 41 3322-1000.
    Fixo: (45)9812 2233."
pat = r"\(\d{2}\)\s?\d{4,5}[ -]?\d{4}|\d{2}\s\d{4}[ -]?\d{4}"
print("entrada:", entrada)
print("regex:", pat)
print("saida:", re.findall(pat, entrada))


titulo("4) VALIDAR PLACA MERCOSUL")
entrada = ["ABC1D23", "ABC-1234", "ABC1234", "ABC1D2X", "AB1C2D3", "
    XYZ9A99"]
pat = r"^[A-Z]{3}\d[A-Z]\d{2}$"
print("regex:", pat)
for item in entrada:
    print(f"  {item!r:10} -> {'VALIDA' if re.fullmatch(pat, item) else '
    INVALIDA'}")


titulo("5) EXTRAIR DATAS dd/mm/aaaa")
entrada = "Prova dia 04/09/2026, entrega em 11/9/2026 e feriado 7/09. Sem
```

```
    data 99-99-9999."
pat = r"\b\d{1,2}/\d{1,2}/\d{4}\b"
print("entrada:", entrada)
print("regex:", pat)
print("saida:", re.findall(pat, entrada))


titulo("6) VALIDAR CEP")
entrada = ["81200-100", "81200100", "8120-0100", "8120010"]
pat = r"^\d{5}-?\d{3}$"
print("regex:", pat)
for item in entrada:
    print(f"  {item!r:12} -> {'VALIDO' if re.fullmatch(pat, item) else '
    INVALIDO'}")


titulo("7) SENHA FORTE")
entrada = ["abc123", "SenhaForte1", "SENHA123", "senha123", "S3nha.Forte!"
    ]
pat = r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d).{8,}$"
print("regex:", pat)
for item in entrada:
    print(f"  {item!r:15} -> {'OK' if re.fullmatch(pat, item) else 'FALHOU
    '}")


titulo("8) REMOVER ESPACOS EXTRAS")
entrada = "A  aula    de   Regex   sera  as   10h,    na sala   B-2.   "
pat = r"\s+"
print("entrada:", repr(entrada))
print("regex:", pat, "(re.sub por espaco, depois strip)")
print("saida:", re.sub(pat, " ", entrada).strip())


titulo("9) EXTRAIR URLs")
```

```
124 entrada = (
125     "Veja https://www.devmedia.com.br/iniciando-expressoes-regulares/6557
    "
126     "e http://example.com.br/aula?x=1. Site: www.uol.com.br"
127 )
128 pat = r"https?://[\w.-]+\.\w+(?:[/?#][^\s.,;]*)?"
129 print("entrada:", entrada)
130 print("regex:", pat)
131 print("saida:", re.findall(pat, entrada))
132
133
134 titulo("10) MASCARAR CARTAO")
135 entrada = "Cartao 5123 4678 9012 3456 aprovado; outro 9876-5432-1098-7654
    negado."
136 pat = r"(?<!\d)(\d{4})[ -]?(\d{4})[ -]?(\d{4})[ -]?(\d{4})(?!\d)"
137 print("entrada:", entrada)
138 print("regex:", pat)
139 print("saida:", re.sub(pat, r"**** **** **** \4", entrada))
```

REFERÊNCIAS

DevMedia. **Iniciando em expressões regulares**. [S. l.]: DevMedia, s.d. Disponível em: https://www.devmedia.com.br/iniciando-expressoes-regulares/6557. Acesso em: 26 set. 2026.

Python Software Foundation. **re: regular expression operations**. [S. l.]: Python Software Foundation, s.d. Disponível em: https://docs.python.org/3/library/re.html. Acesso em: 26 set. 2026.